FIAP – FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
CURSO DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Gustavo de Jesus – RM567926

Gustavo Siciliano – RM568419

Samuel Keniti – RM567614

Manuel Pedro – RM567458

Vinicius Morrone – RM566884

ESP32SAFE: COFRE ELETRÔNICO COM ESP32 E HIVEMQ

São Paulo

2026

SUMÁRIO

1 IDENTIFICAÇÃO 3

2 TÍTULO E OBJETIVO 4

3 FUNCIONAMENTO 5

4 COMPONENTES E LIGAÇÕES 6

5 ADAPTAÇÃO E CÓDIGO-FONTE 8

5.1 Armazenamento da senha e do estado 8

5.2 Esperas e reconexões 8

5.3 Créditos e licença 9

6 ARQUITETURA MQTT 10

7 TESTES 11

7.1 Evidências 12

8 IOT, IOB E INTELIGÊNCIA ARTIFICIAL GENERATIVA 17

9 CONSIDERAÇÕES FINAIS 18

REFERÊNCIAS 19

APÊNDICE A – CÓDIGO-FONTE 20

1 IDENTIFICAÇÃO

Instituição: FIAP – Faculdade de Informática e Administração Paulista.

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas (TDS).

Disciplina: Disruptive Architectures: IoT, IoB & Generative AI.

Professor: Dr. Marcel Stefan Wagner.

Turma: 1TDSPS.

Grupo (identificador MQTT): grupo-k7q2.

Projeto no Wokwi: https://wokwi.com/projects/478258677728303105

GitHub: https://github.com/GustavoSiciliano/CP2_IOT

Integrante (nome completo)	**RM**
Gustavo de Jesus	567926
Gustavo Siciliano	568419
Samuel Keniti	567614
Manuel Pedro	567458
Vinicius Morrone	566884

2 TÍTULO E OBJETIVO

O cofre recebe o nome de **ESP32Safe** (versão 1.0): adaptação, para o ESP32, do projeto “Arduino Electronic Safe” (SHAKED, 2020), com conectividade Wi-Fi e comunicação MQTT com o broker público HiveMQ.

Problema e aplicação. Cofres e armários com senha costumam operar isolados: quem está fora do local não sabe se a trava está acionada. O ESP32Safe controla o acesso localmente (teclado 4×4, LCD 16×2 e servomotor que representa a trava) e publica via MQTT o estado da trava comandada pelo programa, “Locked” ou “Unlocked”, sem expor a senha. Qualquer cliente inscrito no tópico recebe o estado, inclusive quem se conecta depois, graças à mensagem retida.

Delimitação. Não há sensor de posição: o estado publicado é o da trava comandada, não prova que a porta está fechada. A mensagem retida é o último estado conhecido, não um histórico nem prova de que o ESP32 continua conectado. O controle remoto da trava por MQTT não faz parte do escopo.

3 FUNCIONAMENTO

- **Inicialização:** o ESP32 inicializa LCD e ícones, posiciona o servo conforme o estado salvo na flash, inicia o Wi-Fi (SSID “Wokwi-GUEST”) e exibe “Welcome! / ESP32Safe v1.0”. A conexão MQTT ocorre em segundo plano, sem bloquear a interface;
- **Cadastro e confirmação:** sem senha cadastrada, “#” leva ao cadastro. O LCD pede “Enter new code:” e “Confirm new code”, exibindo um asterisco por dígito. Se as entradas coincidem, a senha de quatro dígitos é gravada e o cofre tranca; se diferem, o LCD mostra “Code mismatch / Safe not locked!” e o cofre permanece destrancado;
- **Trancamento e nova senha:** com senha cadastrada, o cofre destrancado mostra “# to lock” e “A = new code”. “#” tranca (servo a 20°, estado gravado, “Locked” publicado); “A” inicia o cadastro de nova senha com confirmação;
- **Desbloqueio e acesso negado:** trancado, o LCD mostra “Safe Locked!” e pede a senha. Se correta, o servo vai a 90°, o LCD exibe “Unlocked!” e “Unlocked” é publicado. Se incorreta, exibe “Access Denied!”, o servo permanece a 20° e nada é publicado;
- **Sincronização:** lock() e unlock() alteram juntos a variável de estado, o servo e o indicador estadoPendente, que garante a publicação da mensagem correspondente (Quadro 1).

Quadro 1 - Correspondência entre os estados do cofre

Estado	Variável (SafeState)	Servo	LCD	Payload MQTT
Locked	_locked = true (flash)	20°	“Safe Locked!”	Locked
Unlocked	_locked = false (flash)	90°	“Unlocked!” / “# to lock”	Unlocked

4 COMPONENTES E LIGAÇÕES

O LCD foi mantido em interface paralela de 4 bits, como na referência (sem I²C). Cores dos fios: vermelho para alimentação (5 V, pino VIN), preto para GND e cores distintas para cada sinal. A numeração de pinos do Arduino UNO não foi reutilizada.

Quadro 2 - Lista de materiais

Componente	Qtd.	Função
ESP32 DevKit V1 (wokwi-esp32-devkit-v1)	1	Controle do cofre e comunicação Wi-Fi/MQTT
Teclado matricial de membrana 4×4	1	Digitação e configuração da senha
LCD 16×2 HD44780 (paralelo)	1	Mensagens de acesso e estado
Servomotor	1	Representação do mecanismo de trava
Mini protoboard e fios	1 / conforme montagem	Barramento de alimentação/GND e interligação

Quadro 3 - Ligações entre os componentes e o ESP32

Componente	Terminal	GPIO do ESP32	Função
LCD	RS / E	GPIO23 / GPIO22	Registrador / enable
LCD	D4, D5, D6, D7	GPIO21, 19, 18, 5	Dados (4 bits)
LCD	VDD e A	VIN (5 V)	Alimentação e luz de fundo
LCD	VSS, V0, RW e K	GND	Terra, contraste, escrita e catodo
Teclado	R1 a R4 (linhas)	GPIO13, 14, 27, 26	Linhas 1 a 4
Teclado	C1 a C3 (colunas)	GPIO25, 33, 32	Colunas 1 a 3
Teclado	C4 (coluna 4)	GPIO16 (RX2)	Coluna 4
Servomotor	PWM (sinal)	GPIO4	Controle (50 Hz)
Servomotor	V+ / GND	VIN (5 V) / GND	Alimentação

O GPIO5 (D7 do LCD) é um pino de inicialização (strapping) do ESP32; seu uso é adequado no simulador e, em geral, em placas reais, pois opera como saída após o boot.

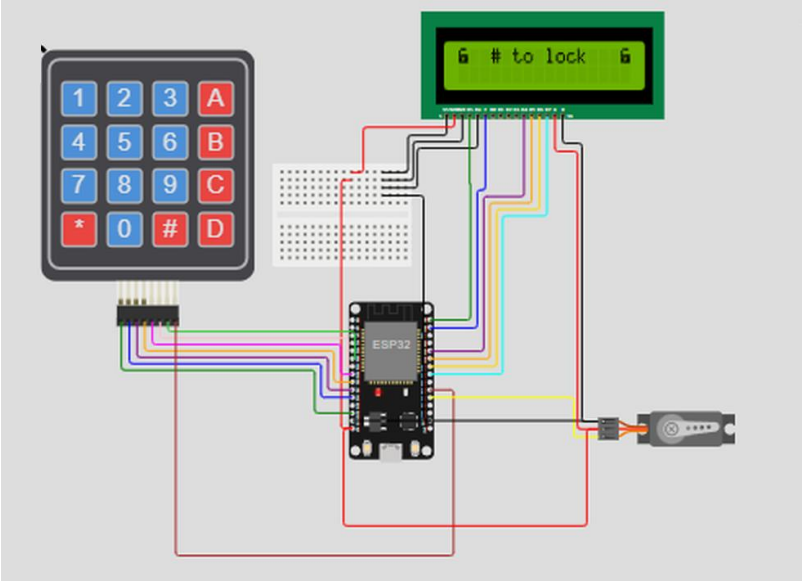


Figura 1 - Circuito completo do ESP32Safe no Wokwi

5 ADAPTAÇÃO E CÓDIGO-FONTE

O código completo e comentado está no Apêndice A. O Quadro 4 resume a migração do Arduino UNO para o ESP32. As bibliotecas usadas são LiquidCrystal (LCD), Keypad (teclado), ESP32Servo (PWM do servo), PubSubClient (MQTT sobre TCP; O'LEARY, [s.d.]) e, do núcleo do ESP32, WiFi.h e Preferences.h.

Quadro 4 - Projeto de referência × ESP32Safe

Aspecto	Referência (Arduino UNO)	ESP32Safe
Pinos	LCD 12 a 7; linhas 5 a 2; colunas A3 a A0; servo 6	GPIOs do Quadro 3; servo no GPIO4
Servo	Servo.h	ESP32Servo (50 Hz; pulso de 500 a 2400 µs)
Senha e estado	EEPROM (classe SafeState)	Preferences (NVS da flash) na classe SafeState
Esperas	delay() e laços bloqueantes	esperar() e lerTecla(), que atendem a rede
Rede	Inexistente	WiFi.h e PubSubClient (publicação retida e reconexão)
Ícones	icons.h	Icons.h e Icons.cpp (createChar, índices 1 a 3)

5.1 Armazenamento da senha e do estado

A classe SafeState manteve a interface original (locked, lock, unlock, hasCode, setCode), mas a EEPROM foi trocada pela biblioteca Preferences, que grava pares chave/valor na NVS (flash), no espaço "cofre", com as chaves "codigo" e "trancado". O método novo begin() lê a senha e o estado salvos; se a senha não tiver quatro dígitos, o cofre inicia sem senha e destrancado. lock() só tranca se houver senha. A senha fica em texto simples (aceitável apenas em simulação) e nunca é enviada por MQTT.

Comportamento após reiniciar: em placa real, senha e estado permanecem na flash: o cofre volta como estava (se trancado, servo a 20°) e republica o estado após reconectar. No Wokwi, porém, a flash não persiste entre reinícios da simulação: o cofre inicia destrancado e sem senha, como registrado no Teste 9.

5.2 Esperas e reconexões

O código de referência usava delay() e laços de espera de tecla que impediriam o atendimento do MQTT (perda do keep-alive). No ESP32Safe:

- servicoRede() é chamada continuamente: refaz o Wi-Fi a cada 10 s, tenta o MQTT a cada 5 s apenas se mqtt.connected() for falso, executa mqtt.loop() e publica se houver pendência;
- esperar(ms) substitui delay(), e lerTecla() chama servicoRede() antes de ler o teclado, em todos os laços de espera;

- estadoPendente é ativado por lock(), unlock() e a cada nova conexão MQTT, e só é desativado quando mqtt.publish() tem sucesso (intervalo mínimo de 1 s entre tentativas);
- a lógica local não depende da rede: sem Wi-Fi ou MQTT o cofre continua operando e nunca destranca por falha de rede. O Client ID do ESP32 é aleatório (“cofre-esp32-” + hexadecimal).

5.3 Créditos e licença

O código preserva o cabeçalho original (“Arduino Electronic Safe, Copyright (C) 2020, Uri Shaked. Released under the MIT License.”); as adaptações estão descritas no cabeçalho do sketch.ino e o texto da licença acompanha a entrega.

6 ARQUITETURA MQTT

O ESP32 é o **publicador**, o HiveMQ público (broker.hivemq.com) é o **broker** e o cliente web do HiveMQ é o **assinante** (OASIS, 2014). O **tópico** é fiap/DA/CP2/1tdsps/grupo-k7q2/cofre/estado; o **payload** é o texto simples “Locked” ou “Unlocked” (sem senha, nome ou RM); a publicação usa **retain**, de modo que o broker guarda o último estado e o entrega a novos assinantes. A publicação do ESP32 usa QoS 0 e a assinatura do cliente web usa QoS 2.

Quadro 5 - Parâmetros de conexão (reprodução dos testes)

Parâmetro	Valor
Wi-Fi no Wokwi	SSID Wokwi-GUEST, sem senha (canal 6)
Host (ESP32 e clientes web)	broker.hivemq.com
ESP32	Porta 1883 (MQTT sobre TCP), sem TLS, sem usuário e senha
Client ID do ESP32	cofre-esp32- seguido de número hexadecimal aleatório
Cliente web 1	https://www.hivemq.com/demos/websocket-client/ – porta WebSocket 8884, SSL/TLS ligado, Client ID clientId-P3KdkYYQtJ
Cliente web 2	Mesmo host, porta 8884, SSL/TLS ligado, Client ID clientId-lu74h7XVfp
Tópico assinado	fiap/DA/CP2/1tdsps/grupo-k7q2/cofre/estado

Procedimento: (a) conectar o cliente web ao host e à porta WebSocket (a porta 1883 do firmware não é a do navegador); (b) assinar o tópico; (c) abrir o projeto no Wokwi e iniciar a simulação; (d) acompanhar o Monitor Serial (“MQTT conectado”, “Enviado: …”) e as mensagens recebidas. Os Client IDs do cliente web são gerados a cada sessão; os indicados acima são os das capturas finais (Figuras 2 e 14).

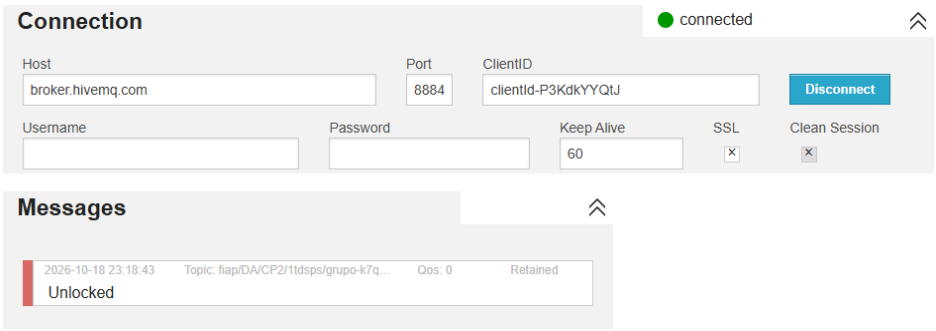


Figura 2 - Cliente web 1 conectado (porta 8884, SSL) com a mensagem retida recebida na assinatura

7 TESTES

Os testes foram executados no Wokwi com o cliente web assinado no tópico do grupo, usando senha fictícia de quatro dígitos. O Quadro 6 reúne procedimento, resultado esperado, resultado observado e evidência; as figuras seguem na seção 7.1.

Quadro 6 - Testes executados

Nº	Procedimento	Resultado esperado	Resultado observado	Evidência
1	Iniciar a simulação com o cliente web já assinado.	Servo e LCD coerentes com o estado inicial; publicação desse estado.	O LCD exibiu “Welcome!” e “# to lock” (sem senha: estado Unlocked, servo a 90°). O serial registrou “MQTT conectado” e “Enviado: Unlocked”. O cliente web recebeu “Unlocked” ao vivo (21:41:54) e a mensagem retida (21:41:37), marcada como Retained.	Fig. 3, 4, 15
2	Com o cofre destrancado, pressionar “#”, cadastrar a senha, confirmá-la e trancar.	Servo trava; “Locked” recebido no cliente web.	O LCD exibiu a transição cadeado aberto → fechado; o serial registrou “Enviado: Locked”; o cliente web recebeu “Locked” (20:31:34) após “Unlocked” (20:31:21).	Fig. 5, 6
3	Com o cofre trancado, informar uma senha incorreta.	Acesso negado; permanece Locked; nenhuma publicação Unlocked.	O LCD exibiu “Access Denied!” e o cofre permaneceu trancado. O serial não registrou novo “Enviado: Unlocked” e o cliente web não recebeu “Unlocked”; a última mensagem era “Locked”.	Fig. 7, 8
4	Informar a senha correta.	Servo libera; “Unlocked” recebido no cliente web.	O LCD exibiu “Unlocked!” e o servo foi a 90°. O serial registrou “Enviado: Unlocked”; o cliente web recebeu “Locked” (23:23:15) seguida de “Unlocked” (23:23:30).	Fig. 9, 10, 11

Nº	Procedimento	Resultado esperado	Resultado observado	Evidência
5	Com o cofre destrancado, pressionar “#”.	Retorno a Locked no dispositivo e no cliente web.	O LCD voltou a “Safe Locked!” e o serial registrou “Enviado: Locked”. O cliente web recebeu “Unlocked” (23:26:22) e, em seguida, “Locked” (23:26:31).	Fig. 12, 13
6	Abrir um segundo cliente web, com outro Client ID, e assinar o mesmo tópico.	Recebimento imediato do último estado retido.	O cliente 2 (clientId-lu74h7XVfp) conectou e recebeu as mensagens do tópico: “Locked” (23:30:53) e “Unlocked” (23:31:08), sem a marca Retained nesta captura. A entrega imediata do estado retido, marcada como Retained, foi registrada na assinatura do cliente 1 (23:18:43).	Fig. 2, 14
7	Provocar perda e restabelecimento do MQTT: digitar “d” + Enter no Monitor Serial, o que aciona mqtt.disconnect() e suspende as tentativas de rede por 20 s.	Controle local preservado; reconexão e republicação do estado atual.	O serial exibiu “Teste: MQTT desligado por 20 segundos”. Durante a queda, servicoRede() apenas retorna e o laço do teclado e do LCD segue em execução. Após o intervalo o ESP32 reconectou e republicou o estado: o cliente web recebeu “Unlocked” às 20:28:32, além da publicação inicial (20:27:33) e da retida (20:27:03).	Fig. 15, 16
8	Digitar uma senha em “Enter new code:” e outra, diferente, em “Confirm new code”.	Erro informado; trancamento não concluído.	O LCD exibiu “Code mismatch / Safe not locked!”. O cofre permaneceu destrancado, o servo não se moveu e nenhuma mensagem “Locked” foi publicada.	Fig. 17
9	Com o cofre trancado, reiniciar a simulação.	Em placa real, senha e estado preservados na flash e republicados após a conexão.	Antes do reinício o cliente web havia recebido “Locked” (23:30:53). Após o reinício, o ESP32 iniciou destrancado e sem senha (LCD “# to lock”, sem “A = new code”) e publicou “Unlocked” (23:31:08). No Wokwi a flash (NVS) não persiste entre reinícios da simulação; em placa real a senha e o estado seriam mantidos.	Fig. 18, 14

7.1 Evidências

Fonte de todas as figuras: capturas de tela do simulador Wokwi, do Monitor Serial e do cliente web HiveMQ (os autores, 2026).

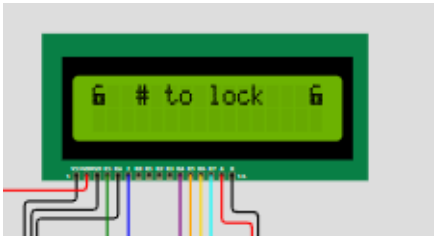


Figura 3 - Teste 1 – LCD no estado inicial

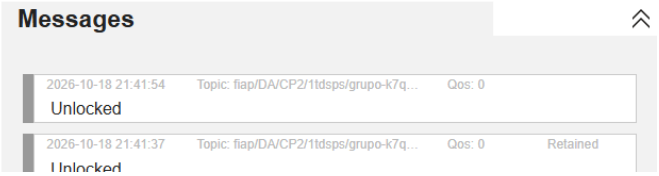


Figura 4 - Teste 1 – “Unlocked” retida e ao vivo no HiveMQ

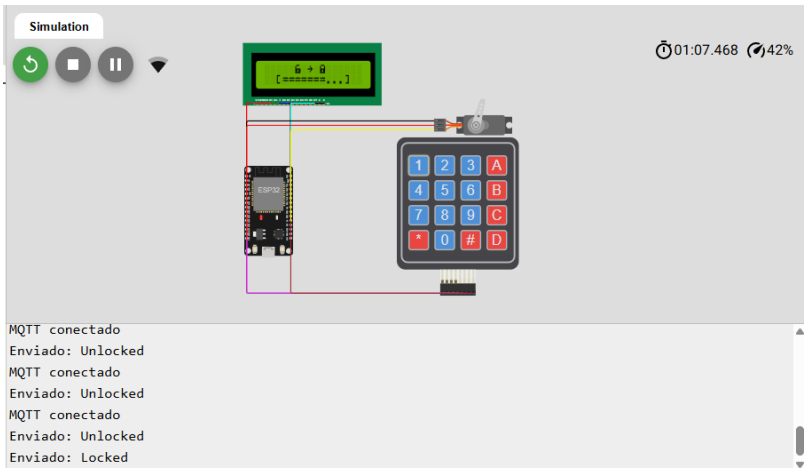


Figura 5 - Teste 2 – LCD e serial com “Enviado: Locked”

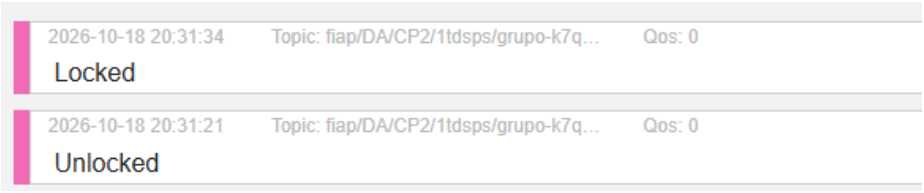


Figura 6 - Teste 2 – “Unlocked” e “Locked” no HiveMQ

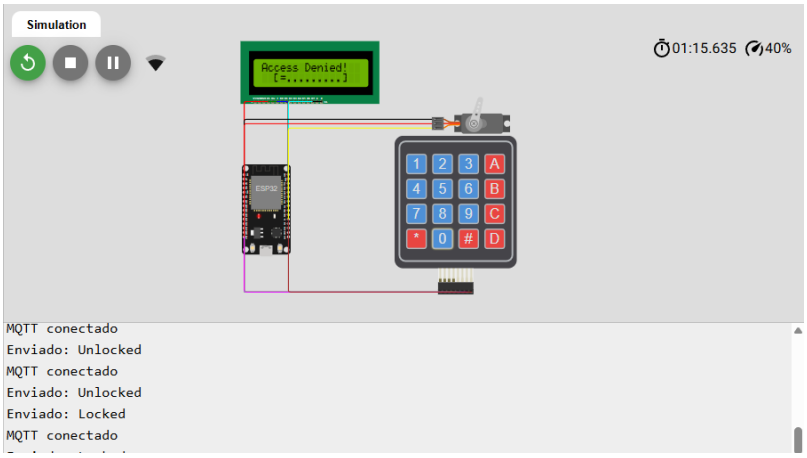


Figura 7 - Teste 3 – “Access Denied!” e serial

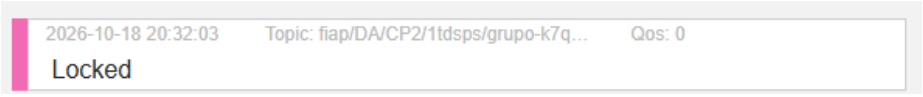


Figura 8 – Teste 3 – estado “Locked” mantido no HiveMQ

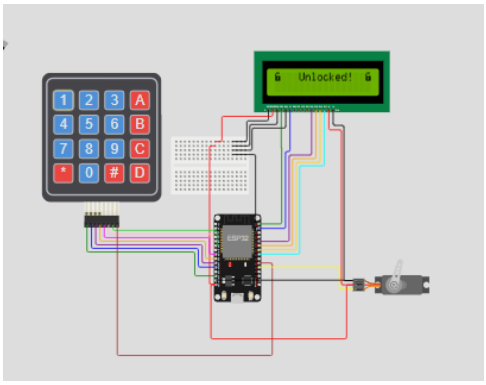


Figura 9 – Teste 4 – LCD “Unlocked!”

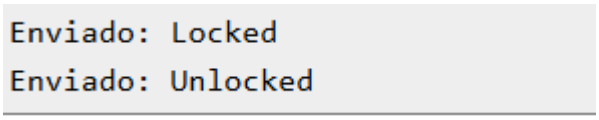


Figura 10 – Teste 4 – serial: “Enviado: Locked” e “Enviado: Unlocked”



Figura 11 – Teste 4 – HiveMQ: “Locked” (23:23:15) e “Unlocked” (23:23:30)

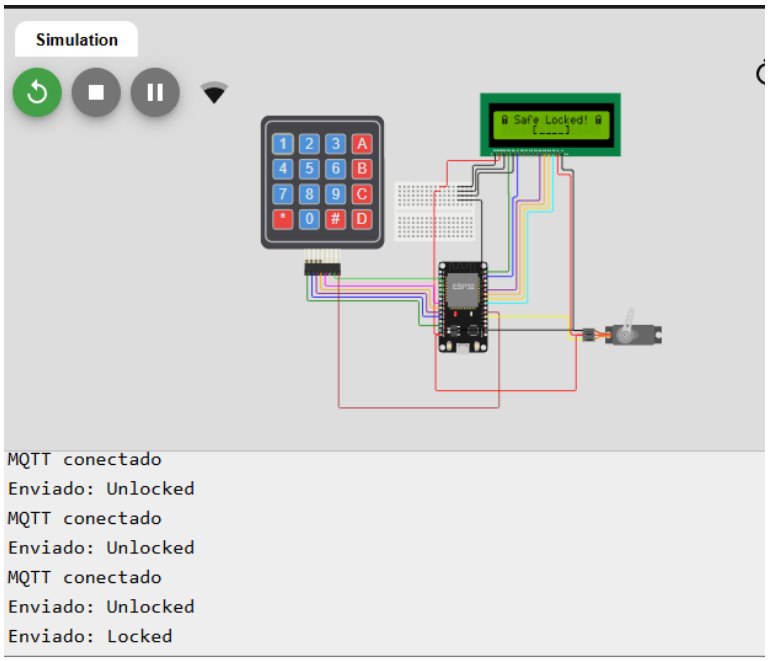


Figura 12 – Teste 5 – LCD “Safe Locked!” e serial com “Enviado: Locked”

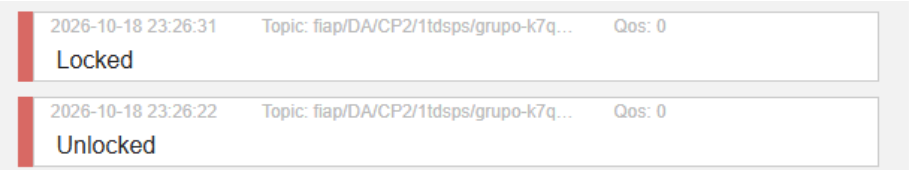


Figura 13 – Teste 5 – HiveMQ: “Unlocked” (23:26:22) e “Locked” (23:26:31)

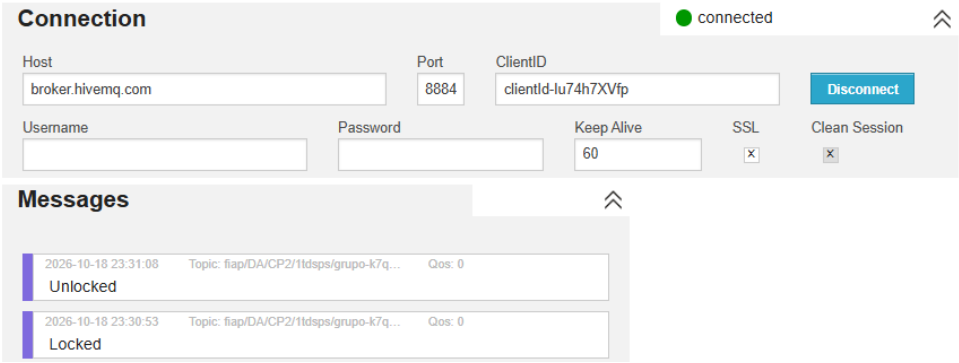


Figura 14 – Teste 6 – cliente web 2 (Client ID distinto) assinado no mesmo tópico



Figura 15 – Teste 7 – serial: conexão, envio e desconexão provocada

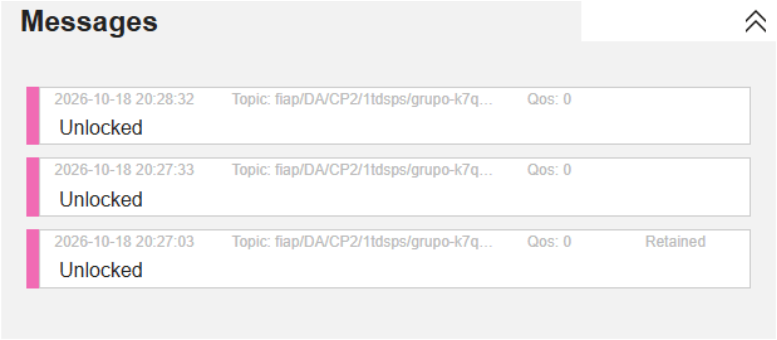


Figura 16 – Teste 7 – mensagens recebidas após a reconexão

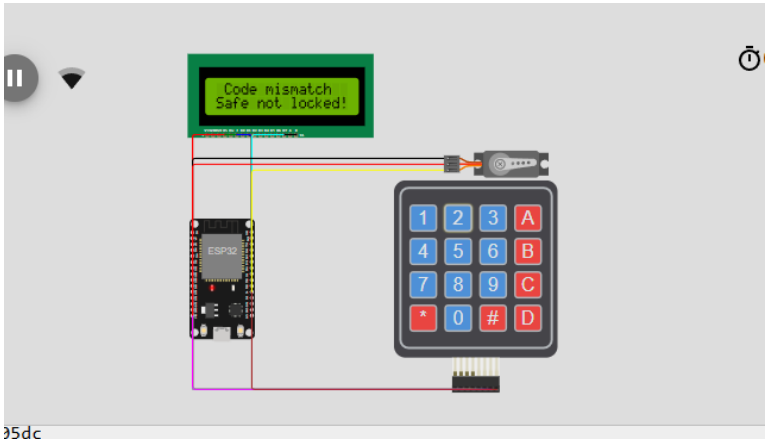


Figura 17 – Teste 8 – senhas não coincidentes

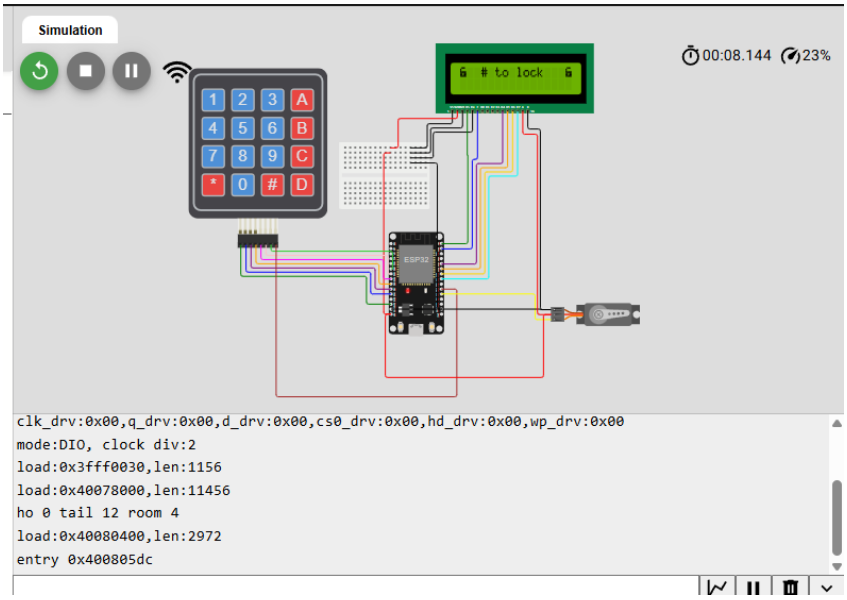


Figura 18 – Teste 9 – ESP32 reiniciado: LCD “# to lock” e boot no serial

8 IOT, IOB E INTELIGÊNCIA ARTIFICIAL GENERATIVA

A) Integração à IoT. O ESP32Safe é um dispositivo com sensor (teclado) e atuador (servo) conectado por Wi-Fi a um serviço em nuvem via MQTT. Como publicador, fica desacoplado de quem consome a informação: qualquer número de assinantes (painel, aplicativo, automação) recebe o estado pelo tópico. A mensagem retida entrega o último estado a quem entra depois, e a republicação após reconexão mantém a nuvem coerente com o dispositivo.

B) Dados adicionais sem a senha. Data e hora dos eventos (NTP); resultado de cada tentativa (sucesso ou falha), contagem de falhas consecutivas e bloqueios, sem o valor digitado; saúde do dispositivo (RSSI, tempo ligado, reinícios, versão do firmware); sensores extras (contato magnético da porta, vibração, temperatura); e o registro de que houve troca de senha, sem seu conteúdo.

C) Padrões analisáveis por IoB. Horários e dias habituais de abertura (linha de base da rotina), frequência de acessos e tempo médio de cofre aberto, anomalias (aberturas fora do horário, sequências de senhas incorretas, acessos em rajada) e mudanças de hábito, com dados de contexto apenas mediante consentimento.

D) Histórico de eventos. Como a mensagem retida guarda só o último estado, o histórico exige um assinante persistente (por exemplo, Node-RED ou função em nuvem) que grave cada evento com data, hora, identificador do dispositivo e tipo em banco de séries temporais ou relacional, a partir de eventos JSON publicados em tópico separado. Consultas agregadas por hora e dia alimentam painéis. Nesse cenário, recomenda-se broker com autenticação e TLS.

E) IA Generativa com privacidade. Um modelo generativo receberia apenas resumos agregados e anonimizados (contagens, faixas de horário, anomalias) para produzir relatórios periódicos, alertas explicáveis e respostas ao proprietário. Para preservar a privacidade: minimizar dados (sem senhas, nomes ou RMs), usar identificadores pseudonimizados, agregar antes de acionar o modelo, obter consentimento e informar a finalidade (BRASIL, 2018), definir retenção e direito de exclusão, criptografar tráfego e armazenamento e manter a decisão final com o ser humano.

Os itens B a E são propostas de evolução e não integram a implementação entregue, que contempla Wi-Fi, MQTT e publicação do estado.

9 CONSIDERAÇÕES FINAIS

O ESP32Safe cumpriu a adaptação do cofre para o ESP32, com controle local por senha e publicação de “Locked” ou “Unlocked” no HiveMQ, em mensagem retida e com republicação após reconexão. Limitações e melhorias futuras: broker público sem autenticação nem TLS (apenas dados de simulação); senha em texto simples na flash (em produto real, hash com sal); estado referente à trava comandada, não à porta (um sensor resolveria); ausência de limite de tentativas incorretas; e persistência da flash não reproduzida no simulador (Teste 9).

REFERÊNCIAS

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

FIAP. **Checkpoint 2: projeto – cofre eletrônico com ESP32 e HiveMQ**. São Paulo: FIAP, 2026. Enunciado da atividade.

HIVEMQ. **HiveMQ WebSocket Client**. Disponível em: https://www.hivemq.com/demos/websocket-client/. Acesso em: 18 out. 2026.

O'LEARY, Nick. **PubSubClient**: Arduino Client for MQTT. [S.l.]: GitHub, [s.d.]. Disponível em: https://github.com/knolleary/pubsubclient. Acesso em: 18 out. 2026.

OASIS. **MQTT Version 3.1.1**. OASIS Standard, 29 out. 2014. Disponível em: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html. Acesso em: 18 out. 2026.

SHAKED, Uri. **Arduino Electronic Safe**. Wokwi, 2020. Disponível em: https://wokwi.com/projects/435037257755377665. Acesso em: 18 out. 2026.

WOKWI. **ESP32 no Wokwi**. Disponível em: https://docs.wokwi.com/pt-BR/guides/esp32. Acesso em: 18 out. 2026.

APÊNDICE A – CÓDIGO-FONTE

Idêntico ao projeto Wokwi (https://wokwi.com/projects/478258677728303105) e à pasta wokwi/ da entrega.

A.1 sketch.ino

```
/**
  Cofre Eletronico com ESP32 + MQTT (HiveMQ)


  Baseado em "Arduino Electronic Safe"
  Copyright (C) 2020, Uri Shaked.
  Released under the MIT License.


  Adaptacoes feitas pelo grupo:
  - Migracao de Arduino UNO para ESP32 (novos GPIOs, ESP32Servo).
  - Senha e estado salvos na flash com Preferences (antes: EEPROM).
  - Conexao Wi-Fi e MQTT (PubSubClient), com publicacao retida do estado.
  - Todos os delay() e laços de espera trocados por funcoes que
    continuam atendendo a rede (esperar() e lerTecla()).
*/


#include <WiFi.h>
#include <PubSubClient.h>
#include <LiquidCrystal.h>
#include <Keypad.h>
#include <ESP32Servo.h>
#include "SafeState.h"
#include "Icons.h"


// Wi-Fi e MQTT
const char* WIFI_SSID  = "Wokwi-GUEST";
const char* WIFI_SENHA = "";
const int   WIFI_CANAL = 6;


const char* MQTT_HOST   = "broker.hivemq.com";
const uint16_t MQTT_PORTA  = 1883;
const char* MQTT_TOPICO = "fiap/DA/CP2/1tdsps/grupo-k7q2/cofre/estado";


WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);
String clientId;


bool estadoPendente = true;
unsigned long ultimaTentativaWifi = 0;
unsigned long ultimaTentativaMqtt = 0;
unsigned long ultimoEnvio = 0;
unsigned long semRedeAte = 0;




// Pinos do ESP32
#define PIN_SERVO 4
```

```
#define LCD_RS 23
#define LCD_E 22
#define LCD_D4 21
#define LCD_D5 19
#define LCD_D6 18
#define LCD_D7 5


#define SERVO_LOCK_POS 20
#define SERVO_UNLOCK_POS 90


Servo lockServo;
LiquidCrystal lcd(LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7);


// Teclado
const byte KEYPAD_ROWS = 4;
const byte KEYPAD_COLS = 4;
byte rowPins[KEYPAD_ROWS] = {13, 14, 27, 26};
byte colPins[KEYPAD_COLS] = {25, 33, 32, 16};
char keys[KEYPAD_ROWS][KEYPAD_COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, KEYPAD_ROWS, KEYPAD_COLS);


SafeState safeState;


// Rede


// Envia o estado atual para o HiveMQ
void publicarEstado() {
  if (millis() - ultimoEnvio < 1000) return;
  ultimoEnvio = millis();


  const char* texto = safeState.locked() ? "Locked" : "Unlocked";
  if (mqtt.publish(MQTT_TOPICO, texto, true)) {
    estadoPendente = false;
    Serial.print("Enviado: ");
    Serial.println(texto);
  }
}


// Teste: digitar 'd' no Serial Monitor derruba o MQTT por 20 segundos
void verificarSerial() {
  while (Serial.available()) {
    if (Serial.read() == 'd') {
      mqtt.disconnect();
      semRedeAte = millis() + 20000;
      Serial.println("Teste: MQTT desligado por 20 segundos");
    }
  }
}
```

```
// Cuida do Wi-Fi e do MQTT. E chamada o tempo todo, inclusive nas esperas.
void servicoRede() {
  verificarSerial();
  unsigned long agora = millis();

  if (agora < semRedeAte) return;

  // Wi-Fi caiu: tenta de novo a cada 10 segundos
  if (WiFi.status() != WL_CONNECTED) {
    if (agora - ultimaTentativaWifi > 10000) {
      ultimaTentativaWifi = agora;
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_SENHA, WIFI_CANAL);
    }
    return;
  }

  // MQTT desconectado: tenta de novo a cada 5 segundos
  if (!mqtt.connected()) {
    if (agora - ultimaTentativaMqtt > 5000) {
      ultimaTentativaMqtt = agora;
      if (mqtt.connect(clientId.c_str())) {
        Serial.println("MQTT conectado");
        estadoPendente = true;
      }
    }
    return;
  }

  mqtt.loop();
  if (estadoPendente) publicarEstado();
}

// Faz o papel do delay(), mas continua atendendo a rede
void esperar(unsigned long ms) {
  unsigned long inicio = millis();
  while (millis() - inicio < ms) {
    servicoRede();
    delay(1);
  }
}

// Le uma tecla e atende a rede ao mesmo tempo
char lerTecla() {
  servicoRede();
  return keypad.getKey();
}

// Trava

void lock() {
  safeState.lock();
  lockServo.write(SERVO_LOCK_POS);
  estadoPendente = true;
```

```
}

void unlock() {
  lockServo.write(SERVO_UNLOCK_POS);
  estadoPendente = true;
}

// LCD e teclado

void showStartupMessage() {
  lcd.clear();
  lcd.setCursor(4, 0);
  lcd.print("Welcome!");
  esperar(1000);

  lcd.setCursor(1, 1);
  String message = "ESP32Safe v1.0";
  for (byte i = 0; i < message.length(); i++) {
    lcd.print(message[i]);
    esperar(100);
  }
  esperar(500);
}

// Le 4 digitos e mostra * no LCD
String inputSecretCode() {
  lcd.setCursor(5, 1);
  lcd.print("[____]");
  lcd.setCursor(6, 1);
  String result = "";
  while (result.length() < 4) {
    char key = lerTecla();
    if (key >= '0' && key <= '9') {
      lcd.print('*');
      result += key;
    }
  }
  return result;
}

void showWaitScreen(int delayMillis) {
  lcd.setCursor(2, 1);
  lcd.print("[.........]");
  lcd.setCursor(3, 1);
  for (byte i = 0; i < 10; i++) {
    esperar(delayMillis);
    lcd.print("=");
  }
}

// Cadastra a senha. Se as duas digitadas forem diferentes, nao tranca.
bool setNewCode() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Enter new code:");
```

```
  String newCode = inputSecretCode();

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Confirm new code");
  String confirmCode = inputSecretCode();

  if (newCode.equals(confirmCode)) {
    safeState.setCode(newCode);
    return true;
  } else {
    lcd.clear();
    lcd.setCursor(1, 0);
    lcd.print("Code mismatch");
    lcd.setCursor(0, 1);
    lcd.print("Safe not locked!");
    esperar(2000);
    return false;
  }
}

void showUnlockMessage() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.write(ICON_UNLOCKED_CHAR);
  lcd.setCursor(4, 0);
  lcd.print("Unlocked!");
  lcd.setCursor(15, 0);
  lcd.write(ICON_UNLOCKED_CHAR);
  esperar(1000);
}

// Logica do cofre

// Cofre destrancado: # tranca, A muda a senha
void safeUnlockedLogic() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.write(ICON_UNLOCKED_CHAR);
  lcd.setCursor(2, 0);
  lcd.print(" # to lock");
  lcd.setCursor(15, 0);
  lcd.write(ICON_UNLOCKED_CHAR);

  bool newCodeNeeded = true;
  if (safeState.hasCode()) {
    lcd.setCursor(0, 1);
    lcd.print("  A = new code");
    newCodeNeeded = false;
  }

  char key = lerTecla();
  while (key != 'A' && key != '#') {
    key = lerTecla();
  }
```

```
  bool readyToLock = true;
  if (key == 'A' || newCodeNeeded) {
    readyToLock = setNewCode();
  }

  if (readyToLock) {
    lcd.clear();
    lcd.setCursor(5, 0);
    lcd.write(ICON_UNLOCKED_CHAR);
    lcd.print(" ");
    lcd.write(ICON_RIGHT_ARROW);
    lcd.print(" ");
    lcd.write(ICON_LOCKED_CHAR);

    lock();
    showWaitScreen(100);
  }
}

// Cofre trancado: pede a senha
void safeLockedLogic() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.write(ICON_LOCKED_CHAR);
  lcd.print(" Safe Locked! ");
  lcd.write(ICON_LOCKED_CHAR);

  String userCode = inputSecretCode();
  bool unlockedSuccessfully = safeState.unlock(userCode);
  showWaitScreen(200);

  if (unlockedSuccessfully) {
    showUnlockMessage();
    unlock();
  } else {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Access Denied!");
    showWaitScreen(1000);
  }
}

void setup() {
  Serial.begin(115200);

  lcd.begin(16, 2);
  init_icons(lcd);

  lockServo.setPeriodHertz(50);
  lockServo.attach(PIN_SERVO, 500, 2400);

  safeState.begin();
```

```
  lockServo.write(safeState.locked() ? SERVO_LOCK_POS : SERVO_UNLOCK_POS);

  clientId = "cofre-esp32-" + String((uint32_t)esp_random(), HEX);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_SENHA, WIFI_CANAL);
  mqtt.setServer(MQTT_HOST, MQTT_PORTA);
  mqtt.setSocketTimeout(2);

  showStartupMessage();
}

void loop() {
  servicoRede();
  if (safeState.locked()) {
    safeLockedLogic();
  } else {
    safeUnlockedLogic();
  }
}
```

A.2 SafeState.h

```
#ifndef SAFE_STATE_H
#define SAFE_STATE_H

#include <Arduino.h>
#include <Preferences.h>

// Guarda a senha e o estado (trancado/destrancado) do cofre.
// Alteracao em relacao ao original: a EEPROM do Arduino foi trocada por
// Preferences (NVS da flash do ESP32). Apos reiniciar, a senha e o estado
// sao lidos de volta da memoria.
class SafeState {
public:
  void begin();
  bool locked();
  void lock();
  bool unlock(String code);
  bool hasCode();
  void setCode(String code);

private:
  Preferences _prefs;
  String _code = "";
  bool _locked = false;
};

#endif
```

A.3 SafeState.cpp

```
#include "SafeState.h"


// Abre o espaco "cofre" e carrega senha e estado salvos.
// Se a senha salva nao tiver 4 digitos,
// comeca sem senha e destrancado.


void SafeState::begin() {
  _prefs.begin("cofre", false);
  _code = _prefs.getString("codigo", "");
  _locked = _prefs.getBool("trancado", false);


  if (_code.length() != 4) {
    _code = "";
    _locked = false;
  }
}


bool SafeState::locked() {
  return _locked;
}


// So tranca se ja houver senha cadastrada. Salva o estado na flash.
void SafeState::lock() {
  if (!hasCode())
  return;


  _locked = true;
  _prefs.putBool("trancado", true);
}


// Compara a senha digitada com a salva. Se estiver certa, destranca e salva.
// Se ja estiver destrancado, apenas retorna true.
bool SafeState::unlock(String code) {
  if (!_locked)
  return true;


  if (code == _code) {
    _locked = false;
    _prefs.putBool("trancado", false);
    return true;
  }
  return false;
}


bool SafeState::hasCode() {
  return _code.length() == 4;
}


// Salva a nova senha (4 digitos) na flash.
// Observacao: a senha fica em texto simples, o que serve para simulacao.
// Ela nunca e enviada pelo MQTT.
```

```
void SafeState::setCode(String code) {
  _code = code;
  _prefs.putString("codigo", code);
}
```

A.4 Icons.h

```
// Icones personalizados do LCD
// Baseado em icons.h do projeto original de Uri Shaked


#ifndef ICONS_H
#define ICONS_H


#include <Arduino.h>
#include <LiquidCrystal.h>


#define ICON_LOCKED_CHAR   1
#define ICON_UNLOCKED_CHAR 2
#define ICON_RIGHT_ARROW   3


// Grava os 3 icones na memoria do LCD. Deve ser chamada depois de lcd.begin().
void init_icons(LiquidCrystal &lcd);


#endif
```

A.5 Icons.cpp

```
#include "Icons.h"


// O HD44780 aceita ate 8 caracteres personalizados
// Aqui usamos os indices 1, 2 e 3, definidos em Icons.h.
// Cada icone tem 8 linhas de 5 pontos
static uint8_t iconeFechado[8] = {
  0b01110, 0b10001, 0b10001, 0b11111, 0b11011, 0b11011, 0b11111, 0b00000
};
static uint8_t iconeAberto[8] = {
  0b01110, 0b10000, 0b10000, 0b11111, 0b11011, 0b11011, 0b11111, 0b00000
};
static uint8_t iconeSeta[8] = {
  0b00000, 0b00100, 0b00010, 0b11111, 0b00010, 0b00100, 0b00000, 0b00000
};


void init_icons(LiquidCrystal &lcd) {
  lcd.createChar(ICON_LOCKED_CHAR, iconeFechado);
  lcd.createChar(ICON_UNLOCKED_CHAR, iconeAberto);
  lcd.createChar(ICON_RIGHT_ARROW, iconeSeta);
}
```